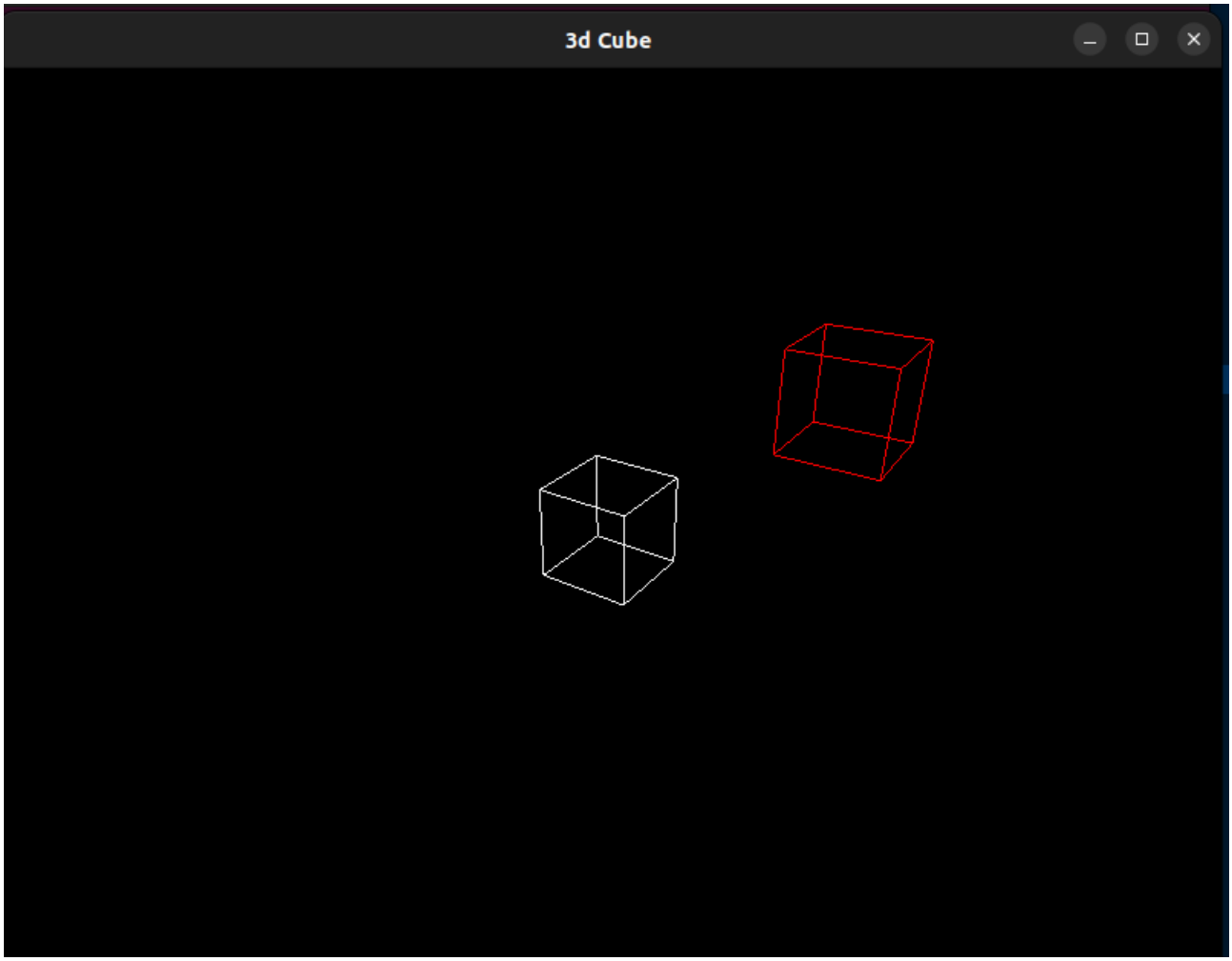
Assignment – 2

Date – 28.03.2026

1. Write an OpenGL program to perform 3D Translation of a Cube and display both the original and translated cube.

```
#include<GL/glut.h>

float cube[8][3] = {
   {-0.5, -0.5, -0.5},
   { 0.5, -0.5, -0.5},
   { 0.5,  0.5, -0.5},
   {-0.5,  0.5, -0.5},
   {-0.5, -0.5,  0.5},
   { 0.5, -0.5,  0.5},
   { 0.5,  0.5,  0.5},
   {-0.5,  0.5,  0.5}
};


float translated[8][3];
float tx = 2.0, ty = 1.5, tz = -1.0;
void translateCube(){
	for(int i=0;i<8;i++){
		translated[i][0] = cube[i][0] + tx;
		translated[i][1] = cube[i][1] + ty;
		translated[i][2] = cube[i][2] + tz;
	}
}
void drawCube(float v[8][3]){
	glBegin(GL_LINE_LOOP);
		glVertex3fv(v[0]);
		glVertex3fv(v[1]);
		glVertex3fv(v[2]);
		glVertex3fv(v[3]);
	glEnd();
	glBegin(GL_LINE_LOOP);
		glVertex3fv(v[4]);
		glVertex3fv(v[5]);
		glVertex3fv(v[6]);
		glVertex3fv(v[7]);
	glEnd();
	glBegin(GL_LINES);
		for(int i=0;i<4;i++){
			glVertex3fv(v[i]);
			glVertex3fv(v[i+4]);
		}
	glEnd();
}
void display(){
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	glLoadIdentity();
```

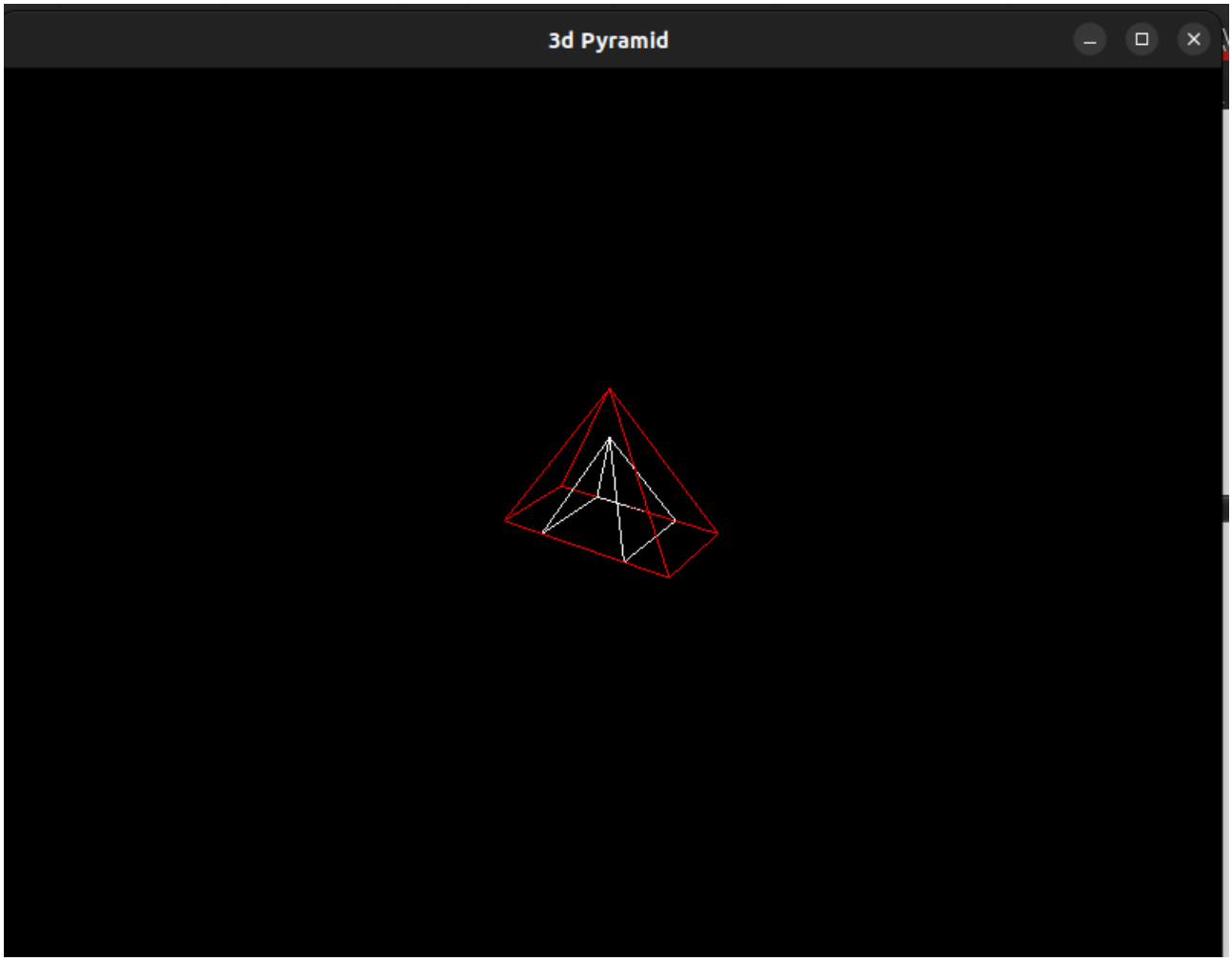
```
	gluLookAt(4, 4, 6, 0, 0, 0, 0, 1, 0);
	glColor3f(1,1,1);
	drawCube(cube);
	glColor3f(1,0,0);
	drawCube(translated);
	glFlush();
	glutSwapBuffers();
}

void init(){
	glEnable(GL_DEPTH_TEST);
	glClearColor(0,0,0,1);
	translateCube();
}
void reshape(int w,int h){
	glViewport(0,0,w,h);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluPerspective(60, (float)w / h, 1, 20);

	glMatrixMode(GL_MODELVIEW);
}
int main(int argc,char** argv){
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGB);
	glutInitWindowSize(800,600);
	glutCreateWindow("3d Cube");
	init();
	glutDisplayFunc(display);
	glutReshapeFunc(reshape);

	glutMainLoop();
	return 0;
}
```


3d Cube

2. Program to perform 3D Scaling of a Pyramid and display both the original and scaled object.

```
#include<GL/glut.h>

float pyramid[5][3] = {
    {-0.5, 0.0, -0.5},
    { 0.5, 0.0, -0.5},
    { 0.5, 0.0,  0.5},
    {-0.5, 0.0,  0.5},
    { 0.0, 1.0,  0.0}
};

float scaled[5][3];


float sx = 2.0, sy = 1.5, sz = 1.0;
void scalePyramid(){
	for(int i=0;i<5;i++){
		scaled[i][0] = pyramid[i][0]*sx ;
		scaled[i][1] = pyramid[i][1]*sy ;
		scaled[i][2] = pyramid[i][2]*sz ;
	}
}
void drawPyramid(float v[5][3]){
	glBegin(GL_LINE_LOOP);
		glVertex3fv(v[0]);
		glVertex3fv(v[1]);
		glVertex3fv(v[2]);
		glVertex3fv(v[3]);
	glEnd();

	glBegin(GL_LINES);
		for(int i=0;i<4;i++){
			glVertex3fv(v[i]);
			glVertex3fv(v[4]);
		}
	glEnd();
}
void display(){
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	glLoadIdentity();
	gluLookAt(4, 4, 6, 0, 0, 0, 0, 1, 0);
	glColor3f(1,1,1);
	drawPyramid(pyramid);
	glColor3f(1,0,0);
	drawPyramid(scaled);
	glFlush();
	glutSwapBuffers();
}
```

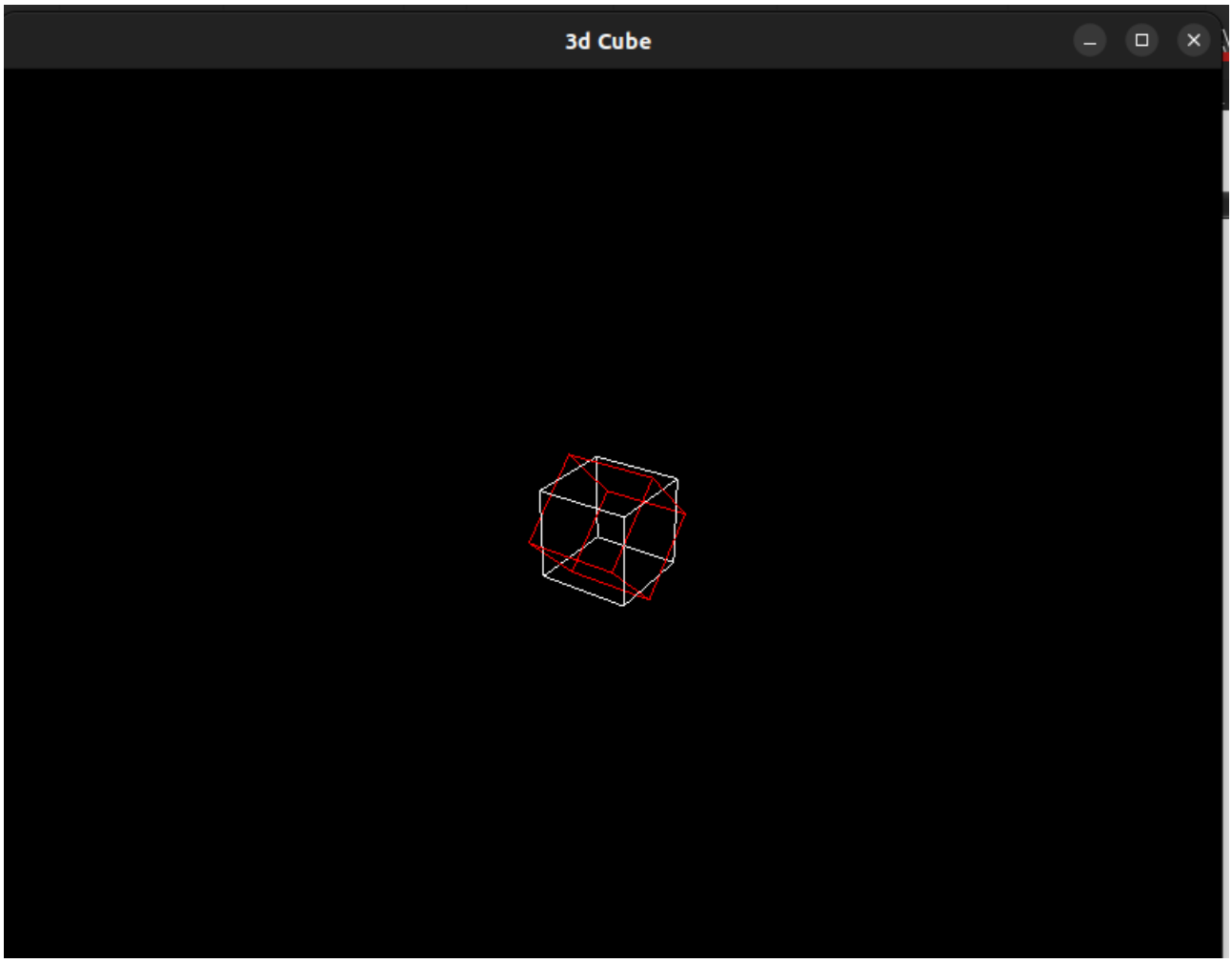
```
void init(){
	glEnable(GL_DEPTH_TEST);
	glClearColor(0,0,0,1);
	scalePyramid();
}
void reshape(int w,int h){
	glViewport(0,0,w,h);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluPerspective(60, (float)w / h, 1, 20);

	glMatrixMode(GL_MODELVIEW);
}
int main(int argc,char** argv){
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGB);
	glutInitWindowSize(800,600);
	glutCreateWindow("3d Pyramid");
	init();
	glutDisplayFunc(display);
	glutReshapeFunc(reshape);

	glutMainLoop();
	return 0;
}
```


3d Pyramid

3. Program to perform 3D Rotation about the X-axis and display the rotated object along with the original.

```
#include<GL/glut.h>
#include<math.h>

float cube[8][3] = {
    {-0.5, -0.5, -0.5},
    { 0.5, -0.5, -0.5},
    { 0.5,  0.5, -0.5},
    {-0.5,  0.5, -0.5},
    {-0.5, -0.5,  0.5},
    { 0.5, -0.5,  0.5},
    { 0.5,  0.5,  0.5},
    {-0.5,  0.5,  0.5}
};

float rotated[8][3];


float angle = 45.0;

float rad;


void rotateX() {
    rad = angle * 3.14159 / 180.0;

    for(int i = 0; i < 8; i++) {
        float x = cube[i][0];
        float y = cube[i][1];
        float z = cube[i][2];

        rotated[i][0] = x;
        rotated[i][1] = y * cos(rad) - z * sin(rad);
        rotated[i][2] = y * sin(rad) + z * cos(rad);
    }
}
void drawCube(float v[8][3]){
        glBegin(GL_LINE_LOOP);
                glVertex3fv(v[0]);
                glVertex3fv(v[1]);
                glVertex3fv(v[2]);
                glVertex3fv(v[3]);
        glEnd();
        glBegin(GL_LINE_LOOP);
                glVertex3fv(v[4]);
                glVertex3fv(v[5]);
                glVertex3fv(v[6]);
```

```
		glVertex3fv(v[7]);
	glEnd();
	glBegin(GL_LINES);
		for(int i=0;i<4;i++){
			glVertex3fv(v[i]);
			glVertex3fv(v[i+4]);
		}
	glEnd();
}
void display(){
	glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
	glLoadIdentity();
	gluLookAt(4, 4, 6, 0, 0, 0, 0, 1, 0);
	glColor3f(1,1,1);
	drawCube(cube);
	glColor3f(1,0,0);
	drawCube(rotated);
	glFlush();
	glutSwapBuffers();
}

void init(){
	glEnable(GL_DEPTH_TEST);
	glClearColor(0,0,0,1);
	rotateX();
}
void reshape(int w,int h){
	glViewport(0,0,w,h);
	glMatrixMode(GL_PROJECTION);
	glLoadIdentity();
	gluPerspective(60, (float)w / h, 1, 20);

	glMatrixMode(GL_MODELVIEW);
}
int main(int argc,char** argv){
	glutInit(&argc,argv);
	glutInitDisplayMode(GLUT_DEPTH | GLUT_DOUBLE | GLUT_RGB);
	glutInitWindowSize(800,600);
	glutCreateWindow("3d Cube");
	init();
	glutDisplayFunc(display);
	glutReshapeFunc(reshape);

	glutMainLoop();
	return 0;
}
```

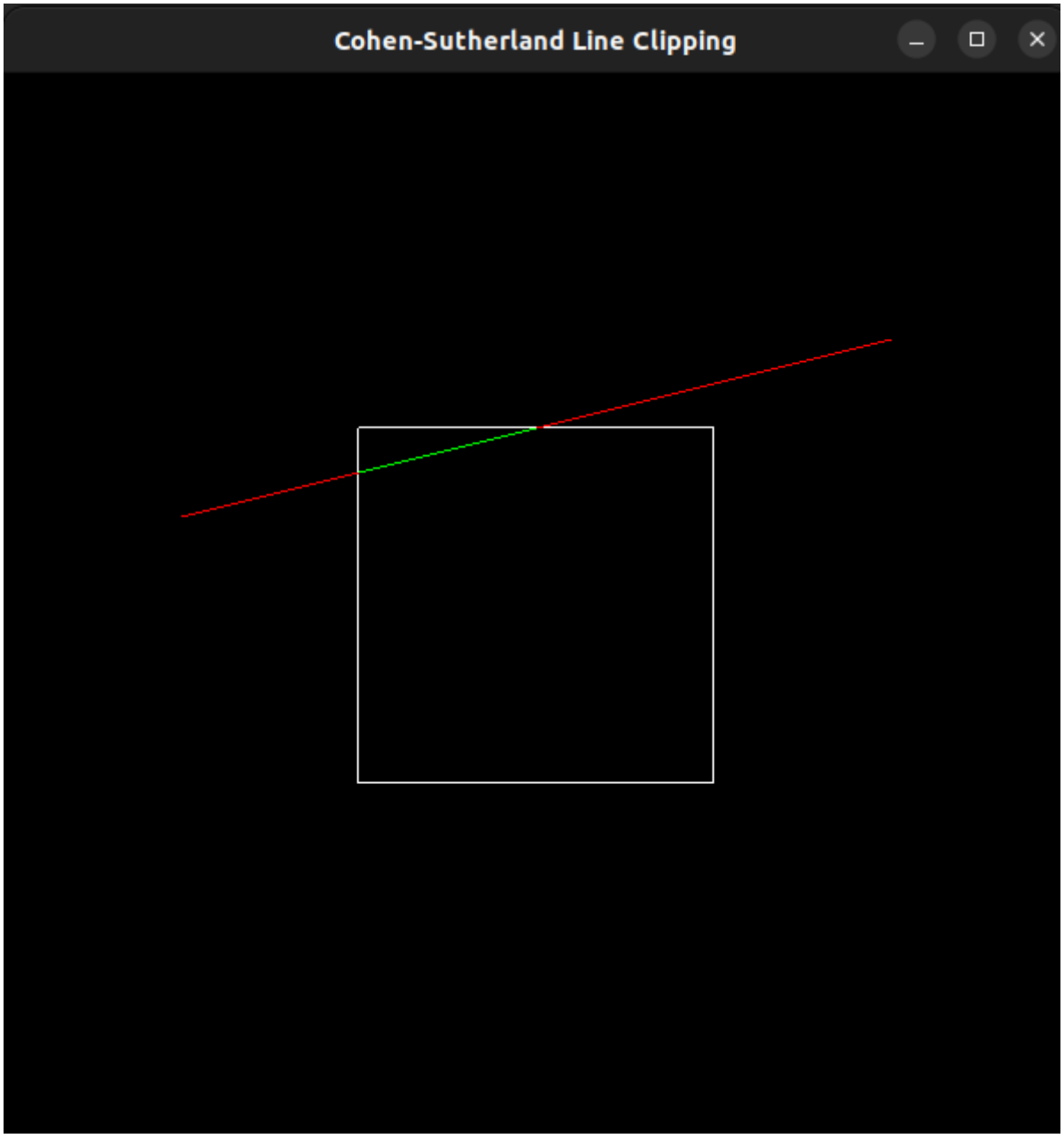

3d Cube

4. Program to implement the Cohen–Sutherland Line Clipping Algorithm to clip a line against a rectangular clipping window.

```
#include <GL/glut.h>
#include <stdio.h>

// Clipping window boundaries
float xmin = -1, ymin = -1, xmax = 1, ymax = 1;

// Line endpoints
float x1 = -2, Y = 0.5;
float x2 = 2, y2 = 1.5;

// Region codes
#define INSIDE 0
#define LEFT   1
#define RIGHT  2
#define BOTTOM 4
#define TOP    8

// Function to compute region code
int computeCode(float x, float y) {
    int code = INSIDE;

    if (x < xmin) code |= LEFT;
    else if (x > xmax) code |= RIGHT;
    if (y < ymin) code |= BOTTOM;
    else if (y > ymax) code |= TOP;

    return code;
}

// Cohen–Sutherland clipping
void cohenSutherland(float *x1, float *y1, float *x2, float *y2) {
    int code1 = computeCode(*x1, *y1);
    int code2 = computeCode(*x2, *y2);
    int accept = 0;

    while (1) {
        if ((code1 == 0) && (code2 == 0)) {
            accept = 1;
            break;
        }
        else if (code1 & code2) {
            break;
        }
        else {
            float x, y;
            int codeOut = code1 ? code1 : code2;
```

```
            if (codeOut & TOP) {
                x = *x1 + (*x2 - *x1) * (ymax - *y1) / (*y2 - *y1);
                y = ymax;
            }
            else if (codeOut & BOTTOM) {
                x = *x1 + (*x2 - *x1) * (ymin - *y1) / (*y2 - *y1);
                y = ymin;
            }
            else if (codeOut & RIGHT) {
                y = *y1 + (*y2 - *y1) * (xmax - *x1) / (*x2 - *x1);
                x = xmax;
            }
            else if (codeOut & LEFT) {
                y = *y1 + (*y2 - *y1) * (xmin - *x1) / (*x2 - *x1);
                x = xmin;
            }

            if (codeOut == code1) {
                *x1 = x;
                *y1 = y;
                code1 = computeCode(*x1, *y1);
            } else {
                *x2 = x;
                *y2 = y;
                code2 = computeCode(*x2, *y2);
            }
        }
    }

    if (!accept) {
        *x1 = *y1 = *x2 = *y2 = 0;
    }
}

// Display function
void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Draw clipping window
    glColor3f(1, 1, 1);
    glBegin(GL_LINE_LOOP);
        glVertex2f(xmin, ymin);
        glVertex2f(xmax, ymin);
        glVertex2f(xmax, ymax);
        glVertex2f(xmin, ymax);
    glEnd();

    // Draw original line (red)
    glColor3f(1, 0, 0);
    glBegin(GL_LINES);
        glVertex2f(-2, 0.5);
        glVertex2f(2, 1.5);
```

```
    glEnd();

    // Perform clipping
    float cx1 = x1, cy1 = Y, cx2 = x2, cy2 = y2;
    cohenSutherland(&cx1, &cy1, &cx2, &cy2);

    // Draw clipped line (green)
    glColor3f(0, 1, 0);
    glBegin(GL_LINES);
        glVertex2f(cx1, cy1);
        glVertex2f(cx2, cy2);
    glEnd();

    glFlush();
}

// Initialization
void init() {
    glClearColor(0, 0, 0, 1);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-3, 3, -3, 3);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Cohen-Sutherland Line Clipping");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
    return 0;
}
```


Cohen-Sutherland Line Clipping